

Freeboard

The verdict is public. The position never is.

Technical Proof of Work for Judges · Midnight Network Wave 1 · September 2026

What Freeboard does

Freeboard is a command-line tool that proves a DeFi lending position is solvent without revealing it. An oracle signs the position; a zero-knowledge circuit on Midnight verifies that signature and computes the health factor entirely in the private domain; and one verdict reaches the chain, SAFE or AT_RISK. Collateral, debt and the liquidation threshold are witnesses. The proof consumes them and nothing writes them anywhere. Most solvency checks ask you to hand over the position and trust whoever reads it. Freeboard hands over the answer and nothing else.

Architecture: the circuit decides, and it checks the signature first

The contract is `contracts/freeboard.compact`, 120 lines of Compact. It declares a single entry point, `checkSolvency(minHealthFactorBps)`, where the threshold is a PUBLIC input chosen by the verifier and everything about the position is private. Two witnesses feed it: `getPosition()` and `getAttestation()`. Before any arithmetic runs, the circuit asserts that a JubJub Schnorr signature over `[collateral, debt, liquidationThresholdBps, asOf]` verifies against `attesterPk`, a ledger field fixed at deployment. Only then does it compute. The health factor test is Aave's, with the basis-point denominators cancelled so it is integer-only with no division and no precision loss:

```
collateral * liquidationThresholdBps >= minHealthFactorBps * debt
```

A prover therefore cannot reach the arithmetic with numbers an oracle has not signed, and the verifier chooses the bar rather than the person being measured.

Two invariants worth a second look

1. A verdict cannot be produced from a position nobody signed

The signature check is not a validation step running alongside the math. It is an assert positioned before it, so an unattested position yields no verdict at all rather than a verdict nobody should trust. There is deliberately no register or rotate circuit for the attester key: an unauthenticated one would let a prover install their own key and self-attest, which turns the entire check back into theater. Rotation is a redeploy until there is a real authority model.

This was tested rather than assumed. The flag `--check --tamper` inflates collateral by a factor of 1000 after the oracle has signed it. The circuit rejects it in 0.4 seconds, against the 45 an accepted proof takes, because the assert fires during circuit execution, before a proof is built and before any transaction exists. The speed is the evidence: a rejection that cost the full 45 seconds would mean the check was happening somewhere later and less trustworthy.

2. The contract has nowhere to put the numbers

The public ledger declares exactly four fields: `attesterPk`, `lastVerdict`, `lastAttestationAt` and `checkCount`. There is no field into which a collateral figure, a debt figure or a threshold could be written, whatever a future circuit attempted. Privacy here is not a policy applied to the data. It is the absence of a place to put it.

One field crosses, on purpose. `asOf`, the oracle's observation stamp, is a private witness that the circuit deliberately discloses to `lastAttestationAt` so a verifier can tell a fresh verdict from a stale one. It sits inside the signed payload, so it cannot be back-dated, and it says nothing about the size of the position. Stating that is more useful than a diagram implying nothing crosses at all.

Verification: bugs caught by actually using it

Five observations, all made by running the software rather than by reading it. Four are bugs. One is a corrected assumption, and it is the most valuable of the five precisely because it was wrong first.

Finding 1: a page that leaked the numbers it claimed to withhold

An earlier dashboard passed a whole `ScenariosResponse` into a client component. Next serialized it into the RSC flight payload, so collateral "1000000", debt "400000", liquidationThresholdBps "8500" and the rendered health factors 2.1250 and 0.9444 all sat in the served HTML. They sat underneath a dot-grid redaction and the sentence "the circuit read these; the ledger never did". Grepping the served HTML found all six. View-source refuted the page's central claim in about four seconds.

Fix: the page is static and no scenario record reaches it. One file holds every value the page states, and the terminal transcript masks the position line as bullets rather than reproducing real figures. Verified by grepping the built output for each of the six numbers and finding none.

Finding 2: a read that reported the state from before the write

The command `--check --read` printed "Accepted. Verdict: SAFE" directly above a frame showing the previous checkCount and no attestation timestamp. `readLedger` goes through `queryContractState` on the indexer, which surfaces a block some time after the chain has it, so the read was racing the write it had just performed. In a demo that reads as the contract not working, which is the worst available way for it to fail. It was found by not being able to explain a number rather than by anything failing: the contract showed one accepted check that no command in that session had produced, and the race turned up underneath it.

Fix (commit af96d7d): `waitForAttestation(asOf)` polls until `lastAttestationAt` equals the `asOf` that was signed. It matches on `asOf` rather than on `checkCount`, so a concurrent call cannot satisfy the wait for a check that has not landed. It returns `settled: false` with the last view read instead of throwing on timeout, because a caller saying "the read is behind" is more useful than an error and far more useful than stale numbers printed as current. And it reads before it sleeps, so a warm devnet costs one query and no delay. Fixed in the shared client rather than in the CLI, because `POST /check` followed by `GET /verdict` is the identical race in the identical two components.

Finding 3: a corrected assumption, not a corrected typo

The indexer began failing with `IndexerQueryError: Invalid response body ... Premature close`, which reads as a network fault. The first diagnosis was that the indexer's compression layer caused it, and a shim was written to force `Accept-Encoding: identity`. That diagnosis was wrong.

Testing each layer separately showed cross-fetch failing for gzip, for br, for zstd, and for the default with no header at all. cross-fetch 4.x loads its own nested node-fetch 2.7.0, which throws `ERR_STREAM_PREMATURE_CLOSE` on any chunked body with no content-length, regardless of encoding. Compression was a correlate rather than a cause; the older ledger-8 indexer simply had not been chunking, which is why this never appeared before.

Correction: the shim delegates to Node's own built-in fetch, which reads the same responses without complaint, and the compression workaround was removed. This is the finding I am most confident is correct, precisely because it survived being wrong first.

Finding 4: a cache from a dead chain that hung instead of failing

A restored wallet sync cache did not error. It hung, for 16 minutes, on a deploy that takes 45 seconds, with zero CPU the entire time, three established websockets to the indexer with empty queues, no error and no timeout. The wallet was waiting for a checkpoint the new genesis had never heard of.

`loadWalletState` validated only that the file parsed and that `version === 1`. The serialized blobs carry no chain marker of their own, so nothing else could have caught it. And because neither compose file declares volumes, every devnet restart is a new chain: this was not an edge case, it was the default after any restart.

Fix (commit a3927f4): the cache is bound to the chain's genesis hash, read via `chain_getBlockHash` at height 0 with no SDK dependency. The binding sits in the type signature rather than in a convention, as `loadWalletState(network, genesisHash, opts)`, so a caller cannot forget it.

Finding 5: a package that passed every check in the repo and could not be installed

`package.json` documents decisions in comment keys prefixed with a double slash. One of them sat inside `dependencies`. npm reads every key of a dependency map as a package name, so the prose became a package spec and any install of the packed tarball died.

Nothing inside the repository could see it. Installing a project never resolves its own dependency map, so `npm install`, the build, the typecheck and the cache tests all passed. The failure appeared only when installing the tarball into a separate directory, and it appeared as an empty npm error: exit 1, no error code line, a log ending mid-manifest-fetch. Reproducing the key in a two-line throwaway package is what finally printed `EINVALIDPACKAGENAME`.

Fix: the comment moved to a top-level key, where it is safe. The general lesson is now recorded in the repository: a green build

proves nothing about installability, and the only proof is packing the tarball and installing it somewhere else. `freeboard-cli@1.0.0` was verified that way before publishing, and again from the public registry afterwards.

A real case, start to finish: the forgery that costs 0.4 seconds

The tamper path is the clearest demonstration in the product, because the cost of the rejection is itself the proof of where the check happens.

POST /check { tamper: true }	
signed.collateral	1,000,000
submitted.collateral	1,000,000,000 <i>(x1000, after signing)</i>
validLocally	false
result	rejected-in-circuit
contract message	"position is not signed by the registered attester"
elapsed	0.4s <i>(an accepted check costs ~45s)</i>

GET /verdict (immediately after)	
lastVerdict	at_risk <i>(unchanged)</i>
checkCount	2 <i>(unchanged)</i>
lastAttestationAt	<i>(unchanged)</i>

→ byte-identical. Nothing was written.

The assert fires during circuit execution. No proof is built, no transaction is submitted, and the ledger is untouched, which is why the rejection is two orders of magnitude cheaper than an acceptance. A system that checked the signature after proving would have produced the same verdict and taken the same 45 seconds, and the difference between those two designs is exactly the difference between a real guarantee and a claim about one.

What a judge can check, and what they cannot

Checkable right now, on any machine with Node 22:

- `npx freeboard-cli@1.0.0 --help` runs from the public npm registry. The package is published and the command it installs is `freeboard`.
- The contract source, the circuit, and the full commit history: 31 commits between 2026-08-28 and 2026-09-09, with the reasoning in the messages rather than in a summary written afterwards.
- The whole path, reproduced locally: bring up the ledger-9 devnet, compile, deploy, check, read, and tamper. Every number in this document came from that path.

Not checkable, and it is worth saying why rather than being caught not saying it:

- There is no public deployment to point at. The contract has only ever run on a local ledger-9 devnet, and neither compose file declares volumes, so the chain a deploy lands on dies with its containers. A contract address from 2026-09-07 exists, but the chain that held it does not, so citing it as verifiable would be dishonest.
- The attester is a mock. Its signing key sits on the same machine as the prover, so the in-circuit check proves the mechanism, not that any particular position is real. Swapping in an independent oracle requires no contract change, which is the whole reason the check was built in-circuit from the start. Public testnet is the next milestone, and it is the one that turns the second bullet from a limitation into a deployment detail.

Why this works: the Plimsoll line

The principle underneath Freeboard is not new and it is not native to crypto. In 1876, after a long campaign by the reformer Samuel Plimsoll, British law required every merchant ship to carry a load line painted on its hull: a disc cut by a horizontal line, marking the depth past which the vessel is overloaded. An inspector reads the waterline against that mark and certifies the ship

is safe to sail without opening a single hold. The cargo stays private. The mark is public. The certification is binding, and it ended a trade in deliberately overloaded ships.

Freeboard is the nautical term for the distance between the waterline and the deck: the margin before water comes over the side. It is literally the health-factor buffer above the liquidation line. The tool publishes the mark and keeps the hold shut, which is the same trade the Merchant Shipping Act made 150 years ago. The mark in the project's own logo is that disc.

Alignment with the Wave 1 judging rubric

Engineering & Implementation — 40%

In-circuit JubJub Schnorr verification positioned before the health-factor arithmetic, so an unattested position produces no verdict; a division-free integer formulation of Aave's health factor that matches the reference implementation exactly; a wallet sync cache bound to the chain's genesis hash in its type signature; and a read-after-write guard that waits on the signed `asOf` rather than on a counter a concurrent call could satisfy. The contract compiles cleanly against Compact 0.34.0 with proving and verifier keys emitted. Published to npm and verified by installation from the public registry, not only by a local build.

Quality Assurance & Reliability — 15%

Five findings above, four of them real bugs caught by actually running the software rather than by reading it: a data-leaking dashboard (retired, not patched), a read-before-write race (fixed and verified live), a cache that hung silently on a dead chain (fixed with a genesis-hash binding), and an uninstallable package that every in-repo check missed (caught only by installing the packed tarball elsewhere, now the project's standing practice before every publish). `test:e2e` and `test:cache` both pass and both assert the property that matters: no position data in public state.

Product & Vision — 15%

Midnight names this exact use case first on its own site: prove solvency without disclosing balances. The demand is real: whales are front-run the moment their health factor slips, and liquidation bots hunt exposed positions. Freeboard is a primitive rather than an application, and the same circuit serves a lender, a DAO treasury committee and an underwriter without modification. The roadmap is scoped honestly: a real attester and a public testnet deployment are the two items standing between this and a claim of "verified," and both are stated as exactly that, not glossed over.

User Experience & Design — 15%

The CLI is the frontend, and it is the one built to be judged on this criterion: it behaves as expected, connects to the real deployed contract end to end, and its output is the actual product experience, not a placeholder for one. It ships as a published npm package, installable and runnable by a stranger with one command. An interactive web dashboard was built and then retired after it was found to leak the exact data it claimed to protect (Finding 1); the decision to cut it, rather than ship a UI that contradicted the product's core claim, is itself a UX judgment the project stands behind. The static landing page states the same limitations as the CLI, in the same voice, so nothing a visitor reads there overclaims what the terminal actually does.

Communication — 10%

The demo's central proof point is not a claim, it is a timing: a forged position rejected in 0.4 seconds where an accepted one takes 45, with a byte-identical ledger either side of it. The limitations are stated in the product itself rather than discovered by a judge, including on the landing page, which says the oracle is a stand-in in the same typeface as everything else.

Business Development & Viability — 5%

The target users are named, not implied: lenders, OTC desks, DAO treasury committees, and underwriters who currently have no way to check solvency without demanding full disclosure. Distribution already exists as a published, installable CLI rather than a repo someone has to build from source. The adoption path is incremental and doesn't require Freeboard to become anyone's primary interface: a real attester and a public deployment are the only two steps between the current mechanism and a version any of those counterparties could rely on today.

Project timeline

Stage	Outcome
Concept	Ported a known domain (Aave health-factor and liquidation math, from a prior risk project) into a privacy primitive, rather than learning a new domain and a new language at once.
Contract first	<code>freeboard.compact</code> written and compiling before any CLI or UI existed. The circuit is the product; everything else exercises it.
Anti-theater pass	Added in-circuit Schnorr verification of an oracle signature, positioned before the arithmetic. A solvency proof over self-reported numbers proves nothing, so this was not optional.
Toolchain	Two version traps cost real time and both named the wrong subsystem: a ledger patch mismatch across compiler, SDK and node, and a fetch failure that presented as a network fault. Both documented with their real causes.
Devnet stability	Genesis-bound the wallet sync cache after a dead-chain cache hung for 16 minutes without erroring. Bounded devnet memory after a guest OOM presented as a Docker crash.
CLI	Rebuilt on clack; 27 fewer packages. Two bugs fixed in the same pass: a leaked LevelDB lock on interrupt, and unknown flags silently ignored, which had the CLI reporting a confident verdict for numbers nobody asked for.
Security pass	External bug hunt run against the CLI before publishing. State moved to the XDG config home, the recovery phrase written 0600 and never printed to stdout, field bounds enforced, signal handling added, and destructive operations gated behind an explicit confirmation variable.
Dashboard, retired	The interactive dashboard was cut after it leaked the numbers it claimed to withhold, could not be reviewed without a fake backend, and depended on a service that binds loopback and holds a signing key. The endpoints are intact and dormant; the decision is scope, not oversight.
Publish	<code>freeboard-cli@1.0.0</code> published after a comment key inside <code>dependencies</code> was found to make the tarball uninstallable while every in-repo check passed. Verified by installing from the public registry.
Landing page	A static page whose only job is the install command, with the limitations stated on it. Rebuilt from scratch after the first attempt; contrast measured against rendered pixels rather than against tokens.

Prepared by

Samuel Urah Yahaya
GitHub: Sammy949
X: @I_am_SamY01
Repository: Sammy949/freeboard
Package: npmjs.com/package/freeboard-cli
September 2026